

Harness 应该越来越薄

Tibo x Dev Interrupted：Scaffolding is coping, not scaling

播客 / 完整逐字稿 • 约 40 分钟 • 会谈日期 2026.01.27

一句话结论

这场最重要的不是“Codex 怎么写代码”，而是 Tibo 对 Agent 系统设计的原则：不要靠越来越复杂的 scaffolding 掩盖模型缺陷；Harness 应该只保留能跨模型代际扩展的 primitive，并随模型变强不断删掉临时拐杖。

四个最值得关注的信号

01 Agent-first 先做一般化 Agent，再让它进入不同产品与场景；产品不是能力边界。	02 垂直整合 Model 与 Harness 要共同迭代；问题可以在环境侧修，也可以进训练侧修。
03 Scalable primitives 不要把 prompt trick、heuristic、compaction 规则堆成不可维护的系统。	04 新瓶颈 生成量上升后，review、正确性与团队协作会迅速成为新瓶颈。

说明：下文将“访谈观点”“技术解读”“方向推演”分开呈现；预测不视为正式产品承诺。

OVERVIEW

访谈背景与章节地图

受访者：Thibault “Tibo” Sottiaux，OpenAI Core Product / Codex 相关负责人。

对谈方：Andrew Zigler（Dev Interrupted / LinearB）

形式：播客 / 完整逐字稿；时长：约 40 分钟

访谈观点

技术解读

方向推演

外部核验

章节地图 / 讨论顺序

时间 / 顺序	主题
00:00	Agentic autonomy 与 “bitter lesson”
约 04:00	Agent-first 与 Model/Harness 垂直整合
约 10:00	为什么 scalable primitives 胜过 clever tricks
15:18	长会话 compaction 为什么暴露 scaffolding 问题
约 20:00	能力跃迁与 capability overhang
25:06	Codex review 成为内部工程工作流的一部分
后段	“super bus factor” 与工程师角色变化

阅读方法

先看 Tibo 明确表达的系统设计原则，再看这些原则对 Agent Runtime、Serving、Sandbox、Memory、Multi-agent 或软件工程流程的含义。不要把方向判断直接当作已经落地的功能。

CORE IDEAS

1. “Scaffolding is coping” 到底是什么意思

- 早期 Agent 常通过大量外部规则补齐模型缺陷：固定 prompt、手工状态机、上下文摘要、重试、工具路由、特殊分支。短期能把成功率顶上去，但这些逻辑会和特定模型行为深度耦合。
- Tibo 的判断更接近 Sutton 的 “Bitter Lesson”：长期真正能吃到模型/算力增长红利的，是简单、可扩展、通用的机制。否则新模型能力已经上去了，旧 Harness 反而成为上限，也就是 capability overhang。
- 一个典型例子是长会话 compaction：为了超过上下文窗口，把旧工作压缩成摘要再重置上下文；但摘要很容易丢掉此前的重要约束。这个问题不是“摘要 prompt 再调一点”就一定值得长期保留，更可能应该让模型和更原生的记忆机制逐步接管。

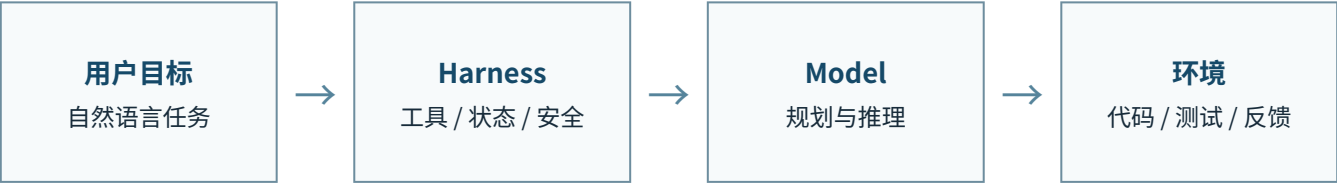
核心判断：这场最重要的不是“Codex 怎么写代码”，而是 Tibo 对 Agent 系统设计的原则：不要靠越来越复杂的 scaffolding 掩盖模型缺陷；Harness 应该只保留能跨模型代际扩展的 primitive，并随模型变强不断删掉临时拐杖。

2. 垂直整合：Agent 不是 Model 外面套一层壳

- Tibo 把 Model 和 Harness 当成同一个 “Agent 系统” 优化。失败案例出现后，团队可以判断：这是模型不会推理？工具接口不清晰？环境反馈不好？还是安全策略太保守？然后选择最合适的层修。
- 这种迭代速度是 frontier lab 的结构性优势：第三方 Harness 往往只能围绕一个黑盒 API 补丁式优化；模型团队则能把高频失败模式直接送回训练、数据和评测。
- 因此未来 Agent benchmark 更应该测整套系统，而不是只看裸模型分数。

SYSTEM VIEW

把访谈观点翻译成系统结构



这里最容易误解的一点

Agent 的“能力”并不等于裸模型能力。对真实任务而言，更接近：

$$\text{Agent Utility} = \text{Model} \times \text{Context} \times \text{Tools} \times \text{Runtime} \times \text{Safety} \times \text{Verification}$$

任何一项接近 0，端到端体验都会坍塌；这也是为什么模型 benchmark 提升并不总能等比例转化为产品收益。

工程上应该关注的闭环

Observe	任务 trace、tool call、权限、失败位置、人工介入、最终结果。
Diagnose	区分是模型、Harness、环境、数据、延迟还是安全策略导致失败。
Improve	把问题放回最合适的层修，而不是默认继续堆 prompt 或规则。
Verify	用可重复 eval 证明端到端成功率、成本和 latency 真正改善。

AI INFRA TAKEAWAYS

对 AI Infra / Agent Infra 的启发

可观测性	需要把失败分解到 model reasoning、tool call、environment、sandbox、memory/ compaction，而不是只有一个“任务失败”。
Harness 预算	每增加一层 heuristic 都会带来 token、延迟、状态复杂度和不可预测交互；应像性能回归一样监控 Harness complexity。
长任务	真正关键的是 checkpoint、可恢复状态、压缩信息损失率和 success criteria，而不是让上下文无限膨胀。
Review 系统	当生成吞吐提升时，验证能力必须同比扩容：测试、静态分析、第二 Agent review、风险动作拦截。
评测	构造 system-level eval：同一模型不同 Harness、同一 Harness 不同模型，区分能力增益来自哪里。

给组内讨论的一个统一框架

把每项 Agent 优化问成四个问题：①端到端成功率提升多少；②wall-clock 缩短多少；③总算力/Token/工具成本增加多少；④风险与人工验收成本是否变高。单看模型分数、Kernel 峰值或单 Token 价格都不够。

$$\begin{aligned} T_{total} = & T_{queue} + T_{prefill} + T_{decode} + T_{tool} + T_{network} \\ & + T_{environment} + T_{verification} + T_{human_wait} \end{aligned}$$

WHAT TO BELIEVE / WHAT TO DO

需要打折看的部分

- 这是 Codex 团队的设计哲学，不意味着所有业务 Agent 都应该“越薄越好”；高度受监管或流程确定的场景仍可能需要显式规则。
- “Bitter Lesson”是长期方向判断，不等于眼前所有 heuristic 都该删除；很多 scaffold 在模型能力成熟前仍然是必要工程手段。
- 逐字稿标注可能存在少量自动转写错误，报告只保留可交叉确认的主线。

给组员的三个可执行动作

1. 给现有 Agent Runtime 画一张 dependency map：哪些逻辑是长期 primitive，哪些只是对当前模型的补丁。
2. 给 compaction / memory 做信息保真评测，而不仅是 token 压缩率。
3. 把 review throughput 当成和 inference throughput 同级的容量规划指标。

本场会谈的最终一句话

这场最重要的不是“Codex 怎么写代码”，而是 Tibo 对 Agent 系统设计的原则：不要靠越来越复杂的 scaffolding 掩盖模型缺陷；Harness 应该只保留能跨模型代际扩展的 primitive，并随模型变强不断删掉临时拐杖。

原始资料

<https://linearb.io/dev-interrupted/podcast/openai-codex-thibault-sottiaux-agentic-autonomy>

整理日期：2026-09-10。报告为技术阅读笔记，不代表 OpenAI 官方路线承诺；产品状态以最新官方文档为准。